

Learning Procedural 3D Stools as Point Clouds

Brown Visual Computing starter project | Ashrith Palla

Can a neural network learn to imitate a 3D shape generator without seeing the generator's parameters?

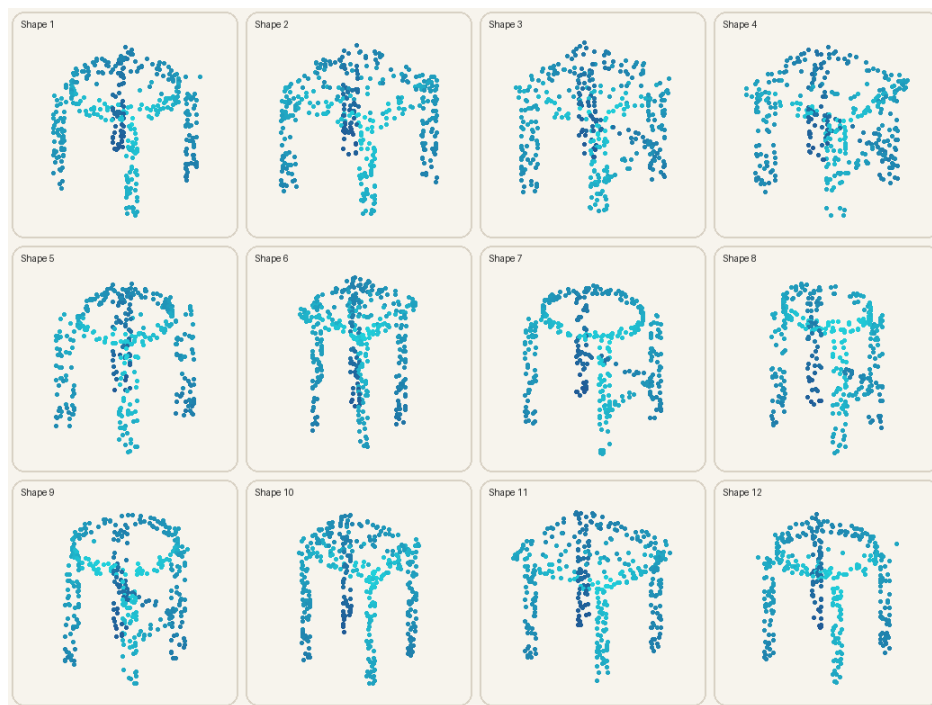


Figure 1. Random stools from the procedural dataset I built from scratch.

I chose stools because they sit in a useful middle ground. They share a clear structure, but small mistakes in the seat, legs, or proportions are immediately visible. I generated the data, trained a PointNet-style autoencoder and VAE, and tested whether the learned space could reconstruct, sample, and smoothly vary the family.

1,000

procedural shapes

384

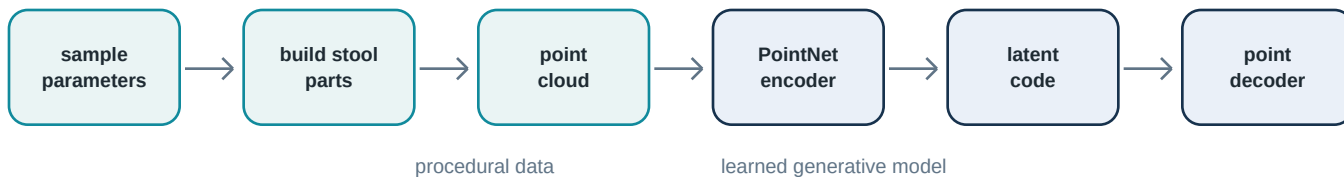
points per shape

40

training epochs

1. Building the world the model would learn

The dataset was not something I downloaded. I wrote a seeded generator that assembles a stool from a seat, four legs, and optional stretchers. I varied width, depth, thickness, height, leg radius, leg spread, seat style, and stretcher presence. The hidden parameters were saved for analysis but never shown to the model.



Why point clouds?

A point cloud is lightweight and directly compatible with point-set networks. It also created the first interesting constraint: points are a set, not a sequence. Point 20 has no fixed semantic meaning, so the encoder applies the same small network to every point and then max-pools the features. That symmetric operation makes the result insensitive to input order.

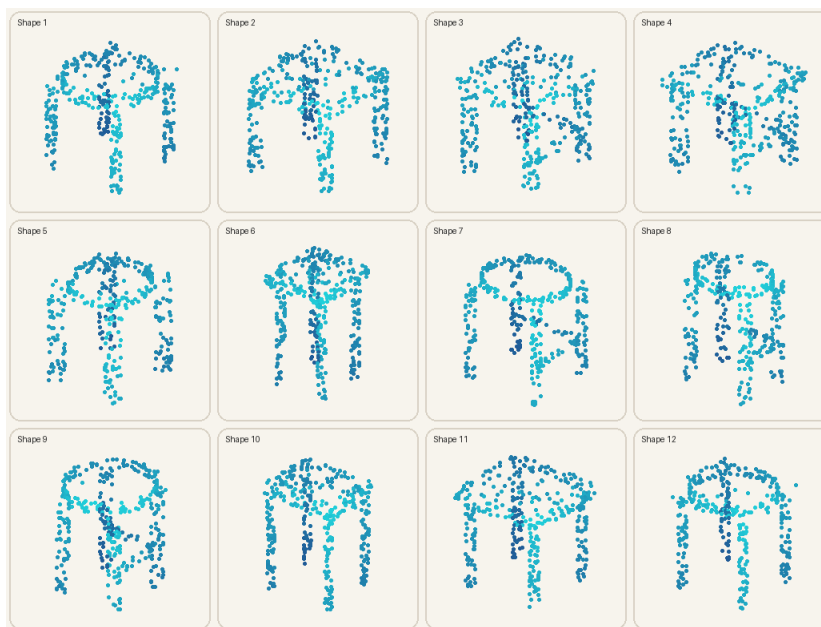


Figure 2. The procedural distribution contains different heights, seat footprints, leg proportions, round and square seats, and optional supports.

A test caught a small but real geometry bug before it reached training. Integer division sometimes assigned fewer points than requested across four legs. I added an explicit remainder fill and kept the test as a reproducibility check.

2. Teaching a network to represent an unordered shape

I started with a deterministic autoencoder. If that model could not reconstruct held-out stools, adding a variational sampling objective would only make the problem harder. The VAE uses the same basic encoder and decoder, but predicts a latent mean and variance. A KL penalty encourages those codes to stay close to a normal distribution so I can draw a new code and decode a new stool.

AUTOENCODER	VARIATIONAL AUTOENCODER
point cloud -> encoder -> z -> decoder -> reconstruction	point cloud -> encoder -> mean, variance -> sampled z -> decoder

Why Chamfer distance?

An ordinary row-by-row coordinate loss would punish two identical surfaces just because their points were listed in a different order. Chamfer distance instead asks how far every point is from its nearest neighbor in the other cloud, in both directions. It is a much better match for sets, although its pairwise distance calculation becomes expensive as point count grows.

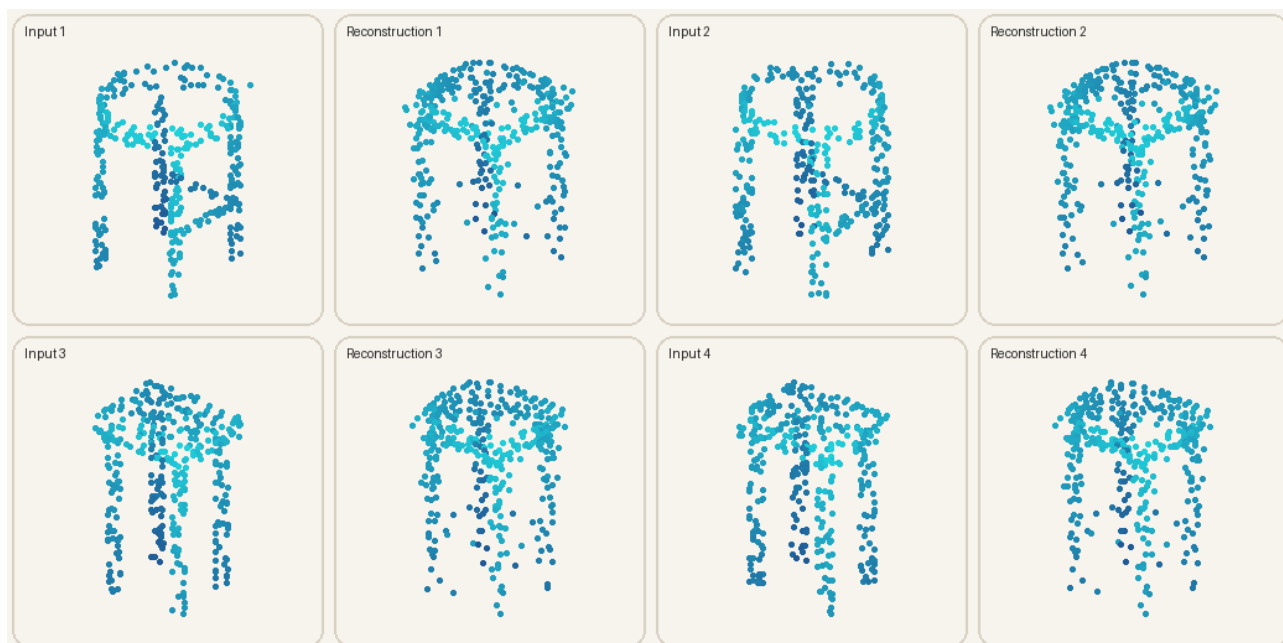


Figure 3. Four held-out inputs paired with VAE reconstructions. The model preserves the main seat and leg structure, while thin supports remain the hardest feature.

The first eight-epoch run was useful but too coarse. For the final run I used 1,000 shapes, 384 points, a 24-dimensional latent code, 40 epochs, and a gradual 12-epoch KL warmup. That warmup let the decoder learn recognizable geometry before the sampling prior became fully active.

3. Final reconstruction results

The final VAE reached a mean held-out Chamfer distance of 0.0223. The matched autoencoder reached 0.0230. The difference is small, so I do not read it as a general claim that VAEs reconstruct better. What matters more is that the VAE kept comparable reconstruction while giving me a latent space I could sample.

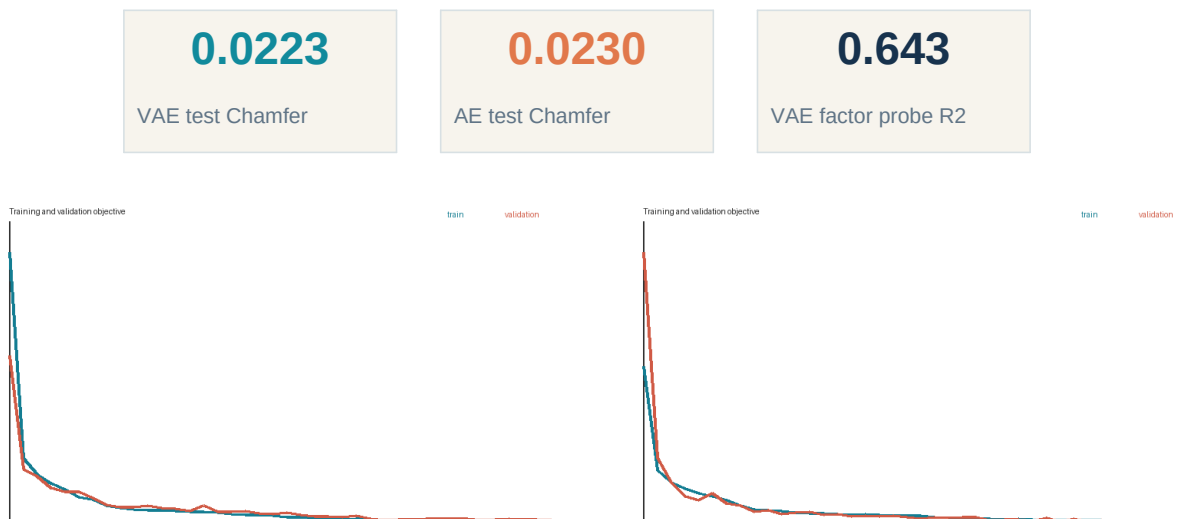


Figure 4. Autoencoder loss, left, and VAE loss, right. Both improve steadily without numerical instability.

A metric that fooled me

My first latent-to-parameter probe reported $R^2 = 1.0$. I was excited for about a minute, then suspicious. There were fewer evaluation examples than latent dimensions, so the regression was underdetermined and the perfect score meant almost nothing. I replaced it with a regularized ridge probe trained on 70 percent of all encoded examples and tested on the remaining 30 percent. The corrected R^2 values, 0.650 for the AE and 0.643 for the VAE, are less dramatic but much more useful.

Training was lightweight on this scale: 15.6 seconds for the 619,032-parameter AE and 13.2 seconds for the 628,272-parameter VAE on Apple MPS.

4. Did random samples actually look like stools?

This was the test I cared about most. A reconstruction begins with a real stool, but a random generation begins with only z sampled from a normal distribution. The final samples have recognizable wide seats and lower supporting regions. Some are clean, while others have incomplete or merged legs. That mixture is important to show rather than hiding the weaker examples.

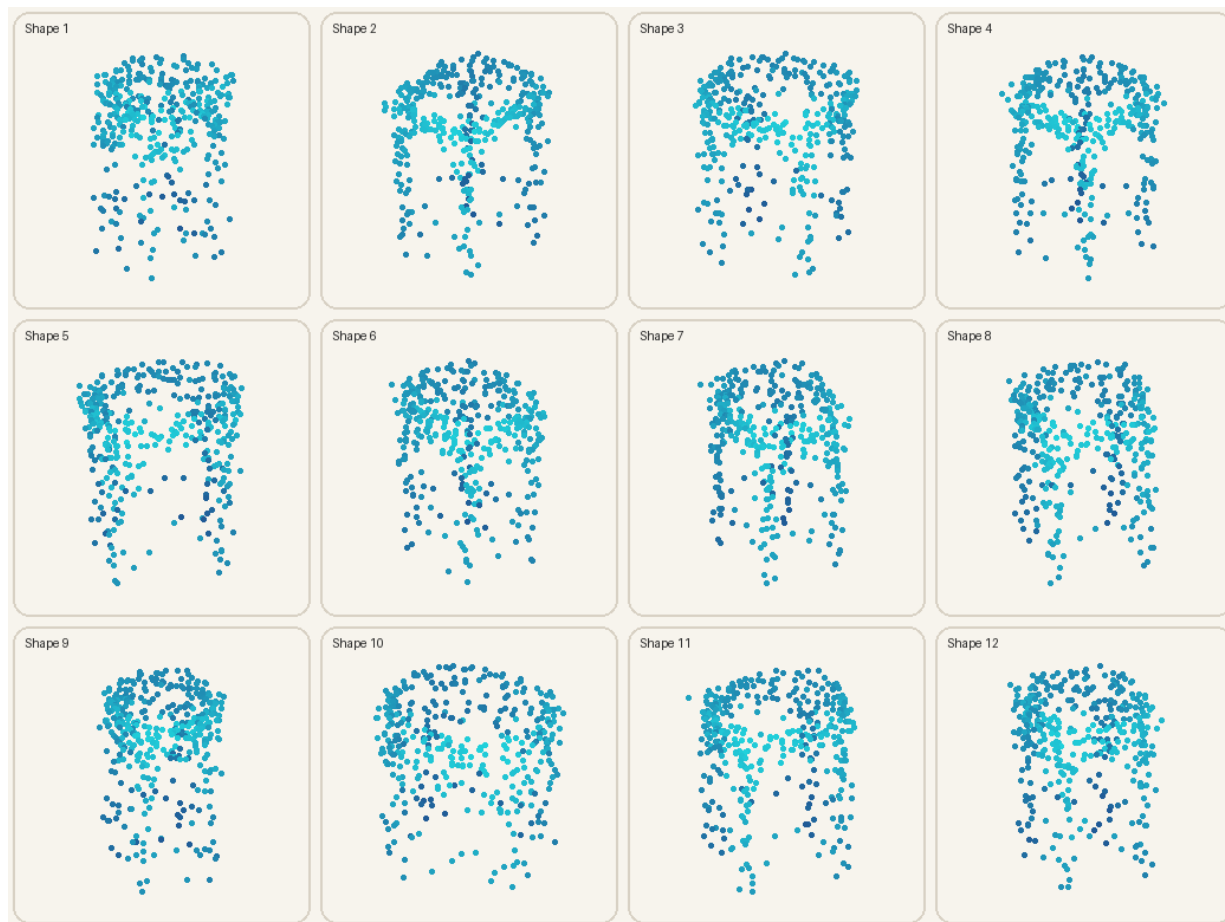


Figure 5. Twelve random prior samples from the final VAE. These are not selected reconstructions.

64 / 64

non-collapsed samples

27 / 64

passed structure check

0.0687

pairwise diversity

I used a deliberately conservative structural check: the cloud needs a wide upper region, a populated lower region, and points reaching floor height. It is better than checking only whether the cloud is nonempty, but it still cannot prove that the object has exactly four connected legs or would stand up physically. The mean nearest-training Chamfer was 0.0673, close to the pairwise diversity of 0.0687, which argues against exact copying.

5. Looking inside the learned shape space

I encoded two separated test examples and linearly moved between their latent codes. The transition stays continuous. Seat proportions and support geometry change gradually rather than disappearing in one step. That does not prove the space is perfectly semantic, but it is encouraging evidence that nearby codes describe nearby shapes.

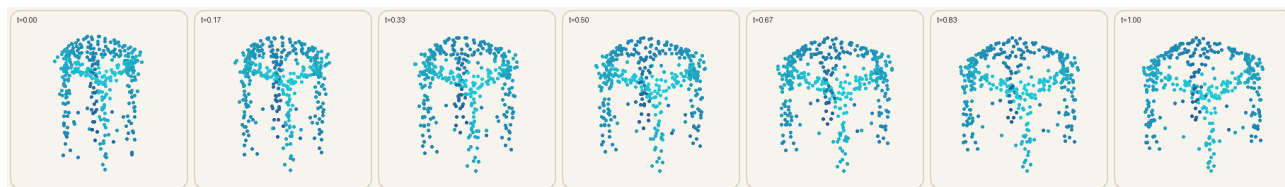


Figure 6. Seven steps between two separated encoded stools.

I also moved along the encoded coordinate with the highest variance. Several properties change at once, which is a useful reminder that an unsupervised latent direction does not have to mean one clean concept such as height or seat width.

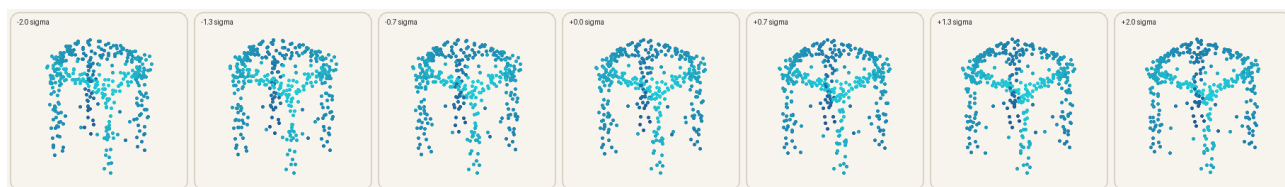


Figure 7. A traversal from -2 to +2 standard deviations along one learned coordinate.

The parameter probe reached R2 around 0.64 to 0.65, so known procedural factors are partly readable from the learned code. I treat that as evidence of accessible geometry information, not as proof of disentanglement or causal control.

6. The experiment that changed my interpretation

I did not want to choose the KL weight beta once and call it finished. I trained three matched settings across seeds 7, 17, and 29. As beta increased, random samples became more diverse and moved closer to the procedural training distribution, but reconstruction error increased. The model was trading one kind of quality for another.

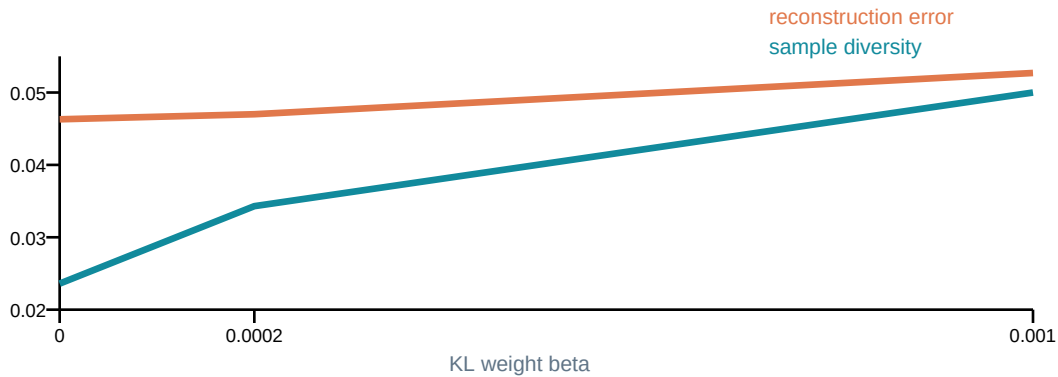


Figure 8. Mean results over three seeds. Orange is reconstruction error, where lower is better. Teal is sample diversity, where higher means samples differ more from one another.

beta	test Chamfer	sample diversity	nearest training
0	0.0463 +/- 0.0087	0.0236	0.1525
0.0002	0.0470 +/- 0.0077	0.0343	0.0940
0.001	0.0527 +/- 0.0101	0.0500	0.0565

What I learned

The project made point-cloud ideas feel concrete. Permutation invariance and Chamfer distance were not small implementation details. They defined what the network was allowed to learn. I also learned to separate reconstruction from generation, to be suspicious of results that look too perfect, and to treat the dataset generator as part of the research rather than background plumbing.

Questions I would ask next

Can a part-aware decoder preserve four distinct legs more reliably? Can a conditional VAE let me request a taller stool or a round seat? If I convert the output to a mesh, can I measure connectivity, watertightness, and whether the stool would actually stand? Those questions feel like the natural next step because they move from visual plausibility toward controllable and physically meaningful geometry.

My main takeaway: a small VAE can learn recognizable structure from an original procedural 3D dataset, but the most interesting work begins where a single loss number stops being enough.